AWS IAM Role for EC2 to S3 Uploads

To Begin with the Lab

Summary of the Lab

In this lab, the difference between the insecure access-key approach and the recommended **AWS IAM role** approach was demonstrated using an **Amazon EC2** web application that uploads files to **Amazon S3**. First, an IAM user was created and given S3 upload permissions through a customer-managed policy, and that user's access key and secret access key were embedded directly into the Flask application running on EC2. The application worked and successfully uploaded a file to the S3 bucket, but the method was shown to be unsafe because the credentials were stored in code. Next, the insecure setup was replaced with an **IAM role** attached to the EC2 instance, and the application was updated to use Boto3 without hardcoded AWS credentials. The same upload flow then worked again, but this time using temporary role credentials, which matched AWS best practices.

Understanding IAM Roles for EC2 Access

An **IAM role** is a temporary identity that AWS services and users can assume when they need permission to do a job. It exists so you do not have to store long-term credentials inside an application or server. In this lab, the EC2 instance needed permission to upload files to S3, and the role gave it temporary credentials automatically. That is safer than using an IAM user's access key and secret key because the application can keep working without exposing secrets in code.

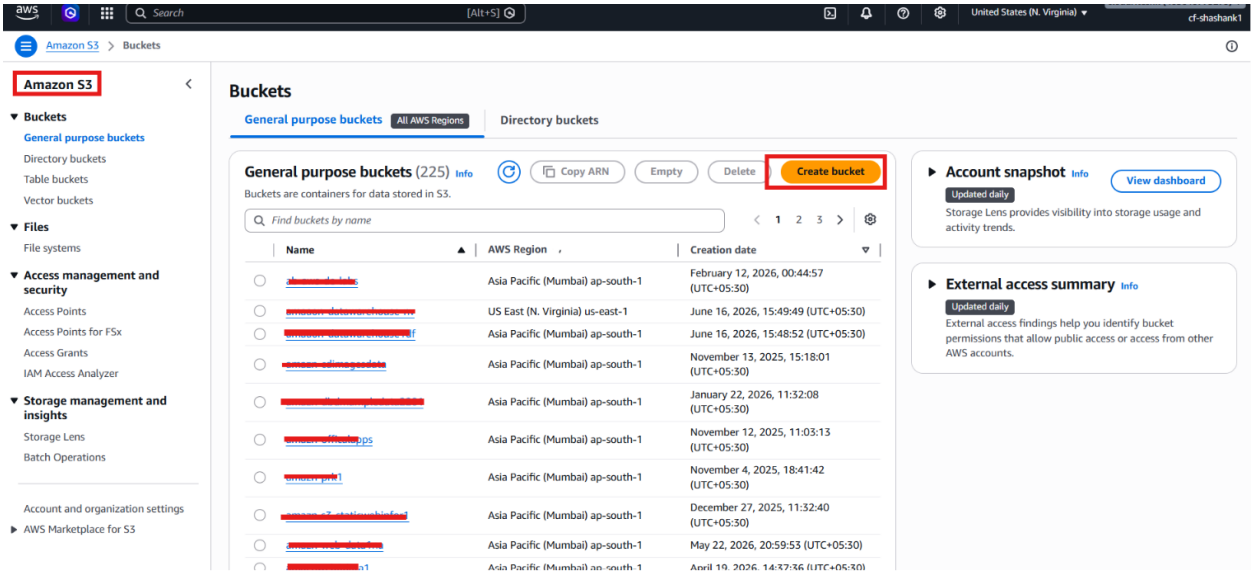
Example:

A Flask app running on an EC2 instance accepts a photo upload and sends it to an S3 bucket named myapp03. In the insecure version, the app contains an access key and secret key for S3AccessUser. In the secure version, the EC2 instance assumes MyAppS3Access as an IAM role and uploads the file without any AWS keys stored in the application.

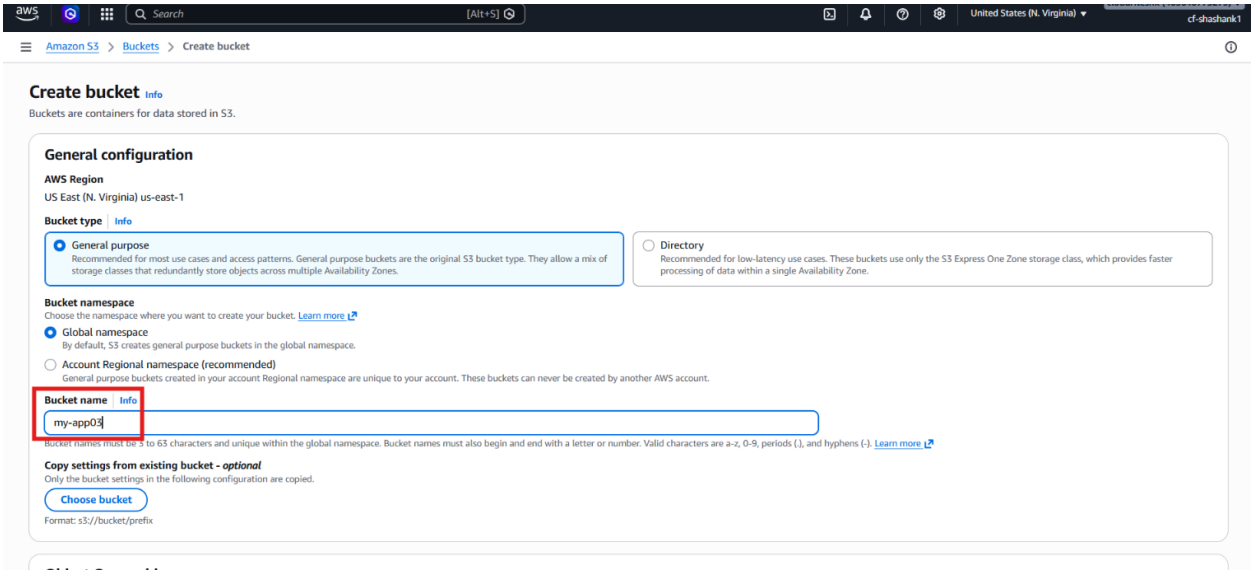
Lab Steps

Step 1 – Create the S3 Bucket

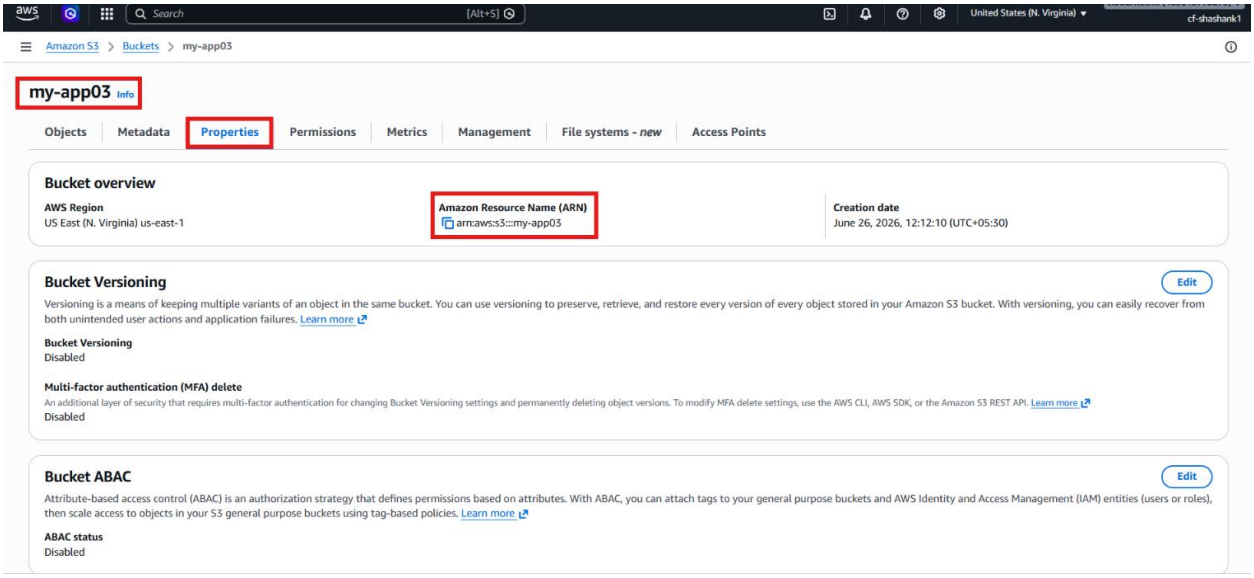
- Sign in to the AWS Management Console.
- Open **Amazon S3**.
- Click **Create bucket**.



- Enter the bucket name: my-app03
- Keep the bucket in the selected AWS Region.



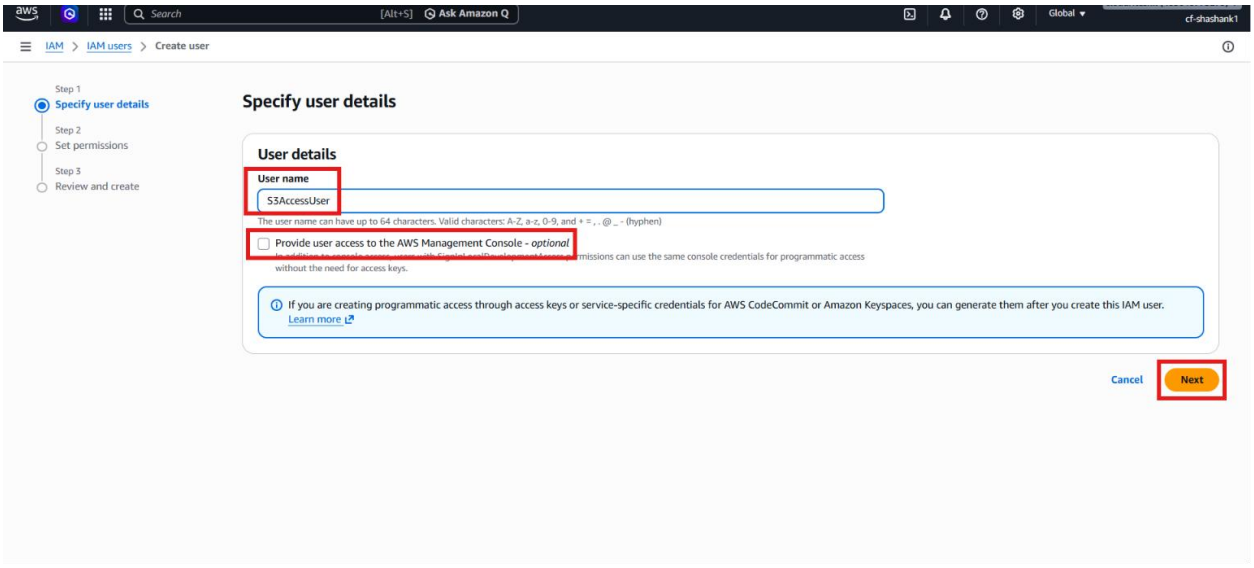
- Create the bucket.
- Open the bucket properties and note its ARN for the policy.



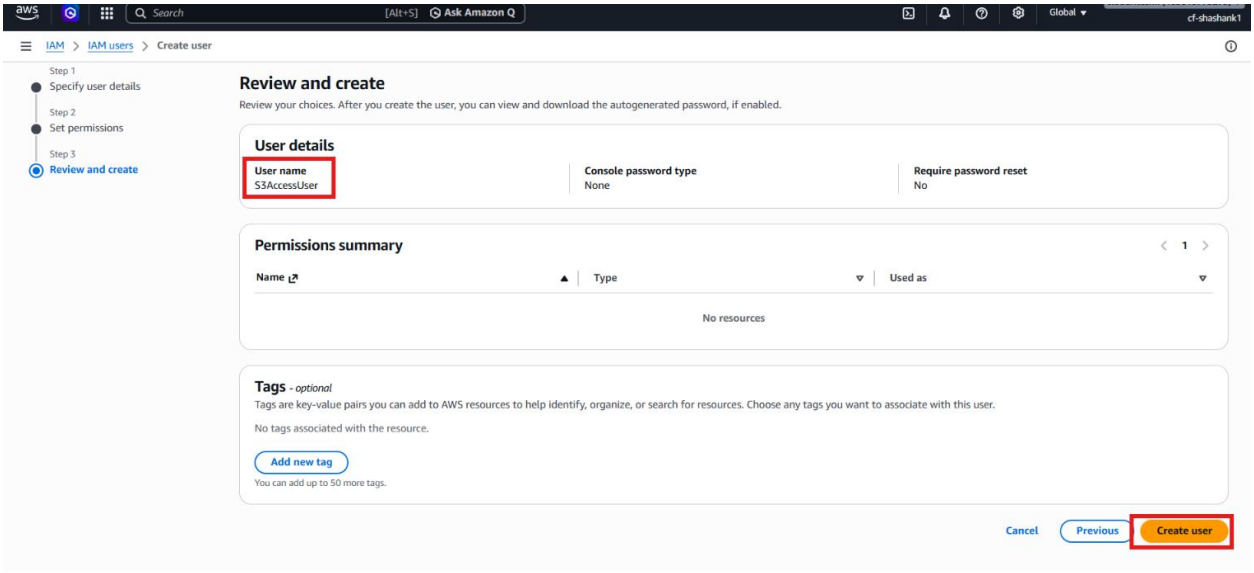
Step 2 – Create the IAM User for the Insecure Demo

- Open **IAM**.
- Click **Users**.
- Click **Create user**.

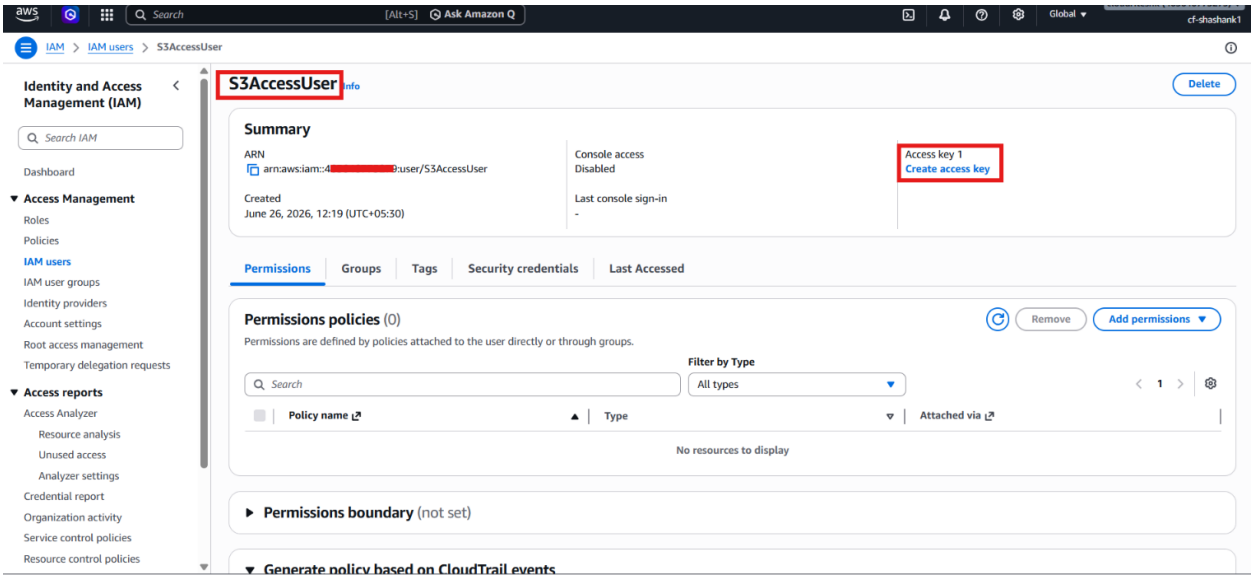
- Enter the username: S3AccessUser
- Do not give this user console access and no permission for now.



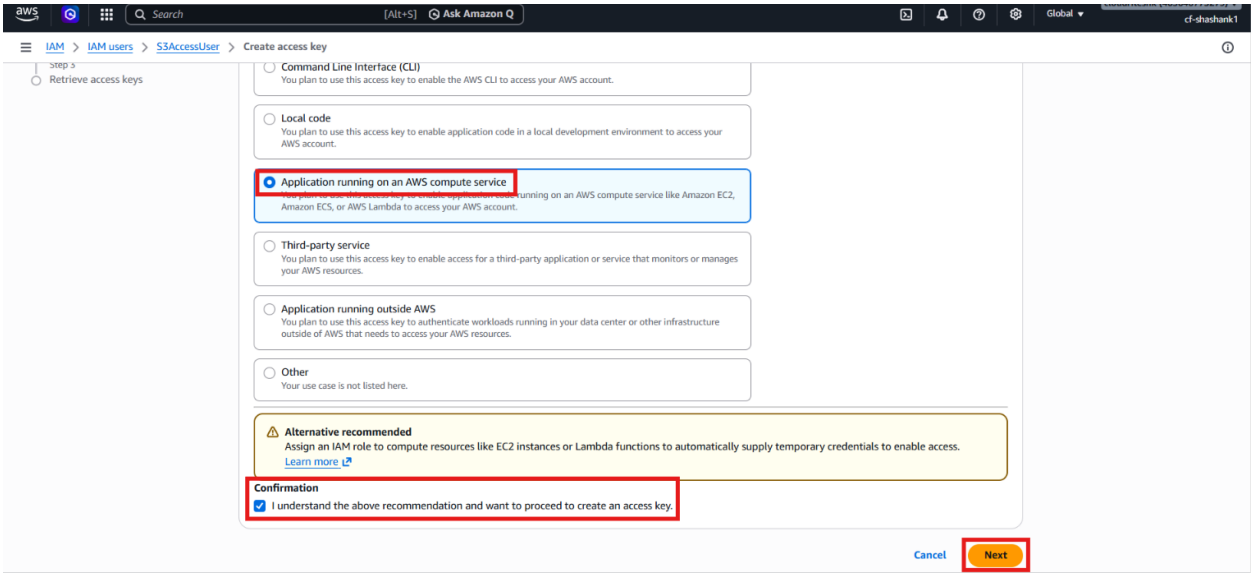
- Create the user.



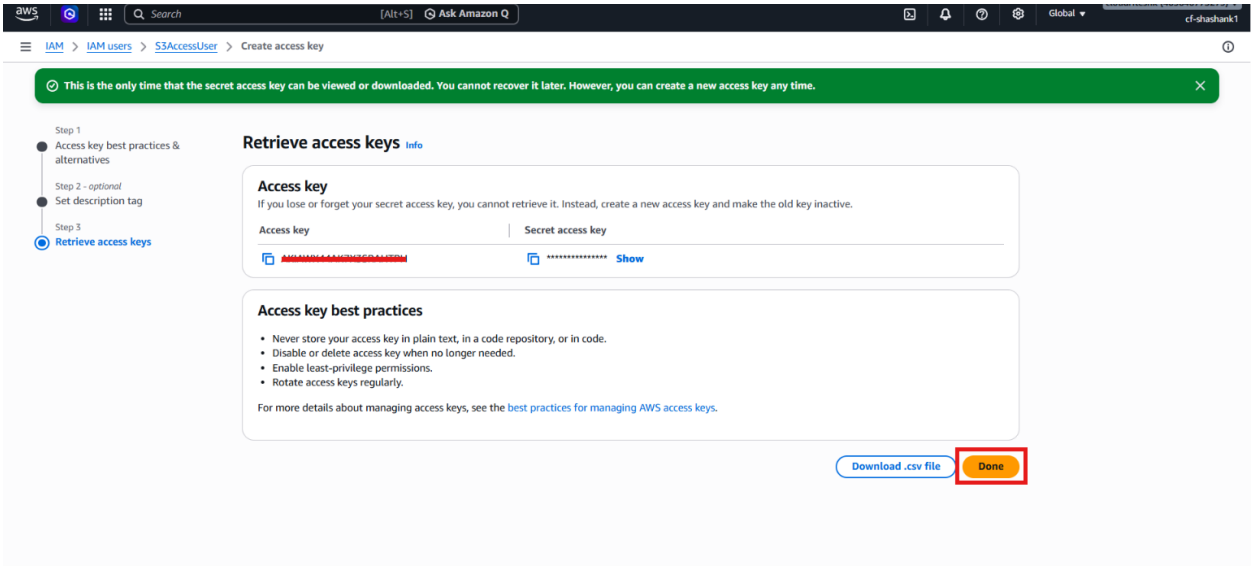
- Open the user and click **Create access key**.



- Select the **Application running on AWS compute service** use case.
- Tick the confirmation.



- Save the access key and secret access key.



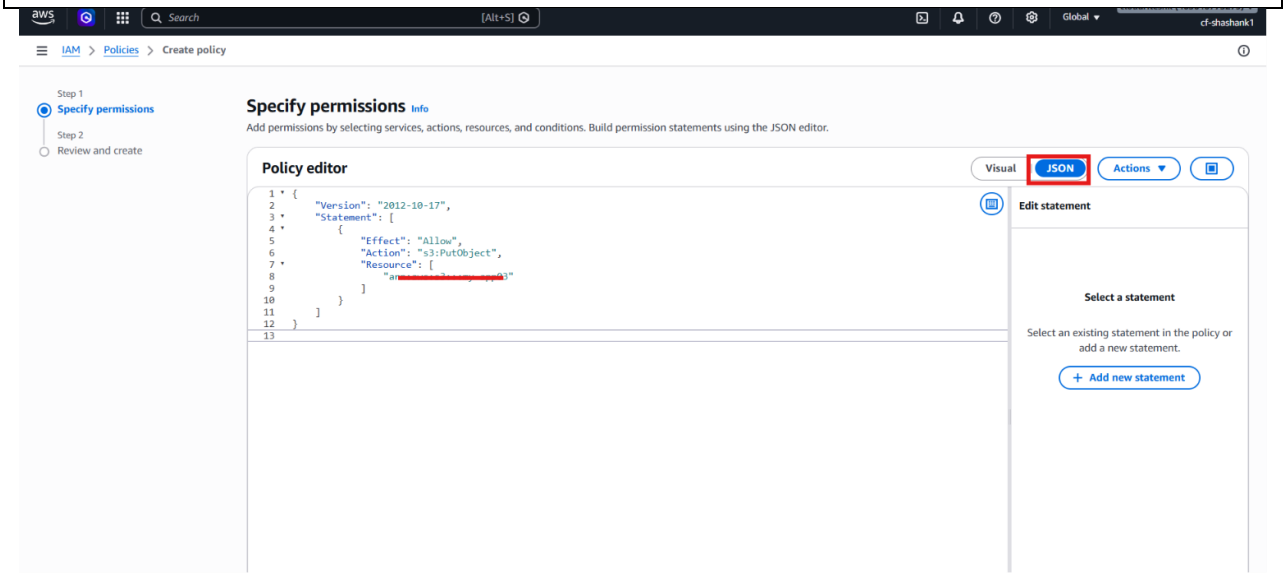
Step 3 – Create the S3 Upload Policy

- Open **IAM → Policies**.
- Click **Create policy**.

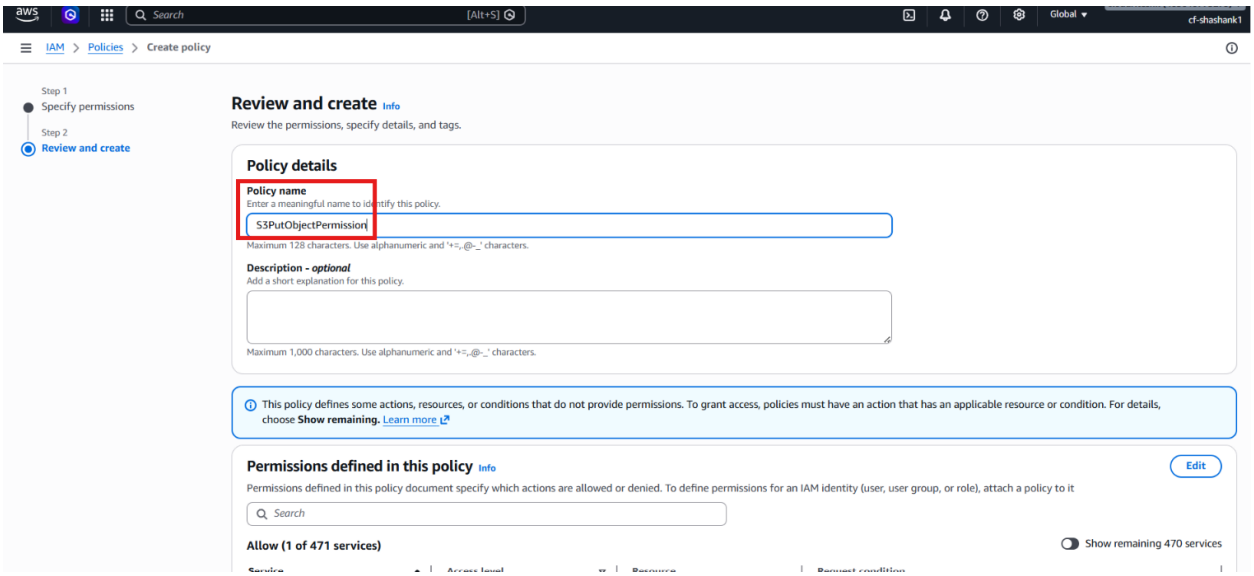
- Switch to the **JSON** tab.
- Paste the S3 upload policy.
- Replace the bucket placeholder with your bucket ARN.

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
```

```
            "Effect": "Allow",
            "Action": "s3:PutObject",
            "Resource": [
                "<YOUR-BUCKET-ARN/>"
            ]
        }
    ]
}
```



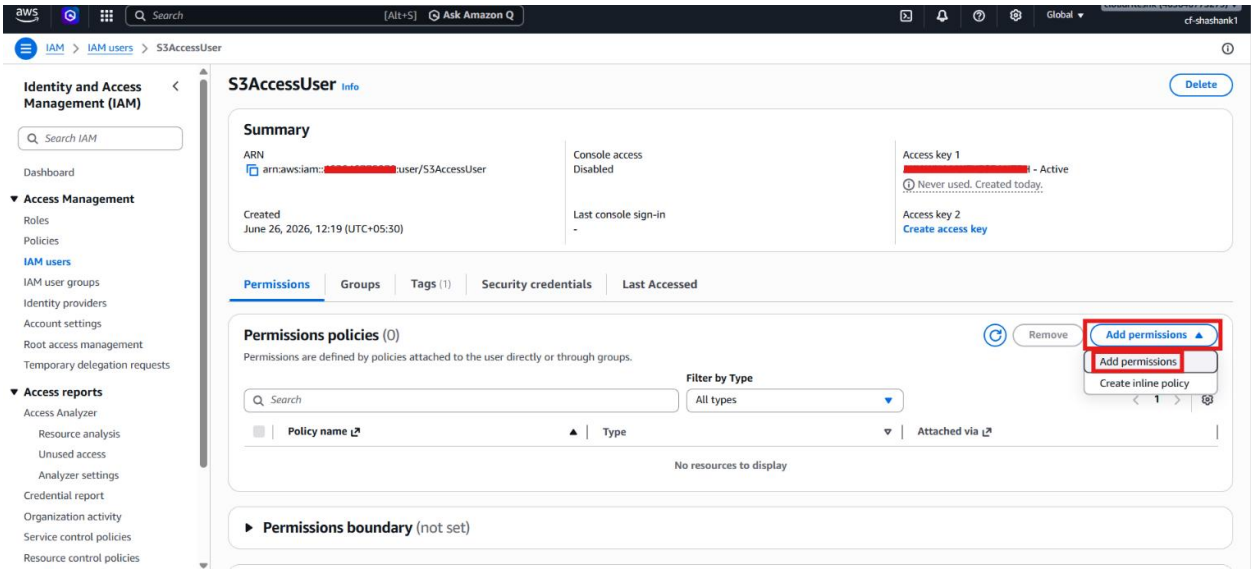
- Name the policy: S3PutObjectPermission
- Create the policy.



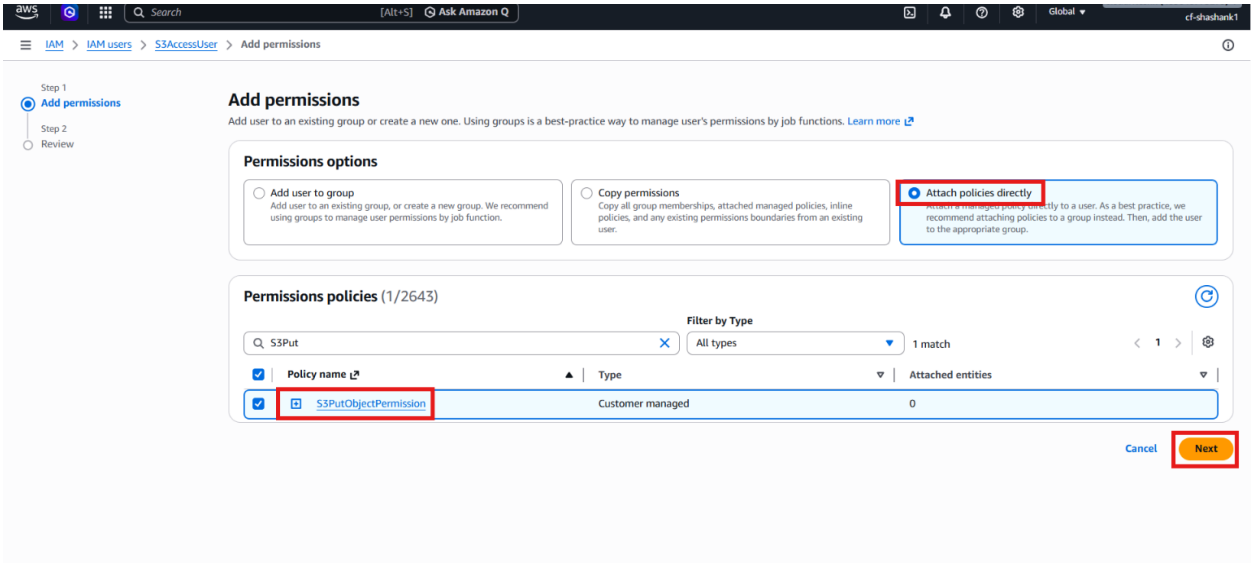
Step 4 – Attach the Policy to the IAM User

- Return to **IAM → Users**.

- Select **S3AccessUser**.
- Click **Add permissions**.

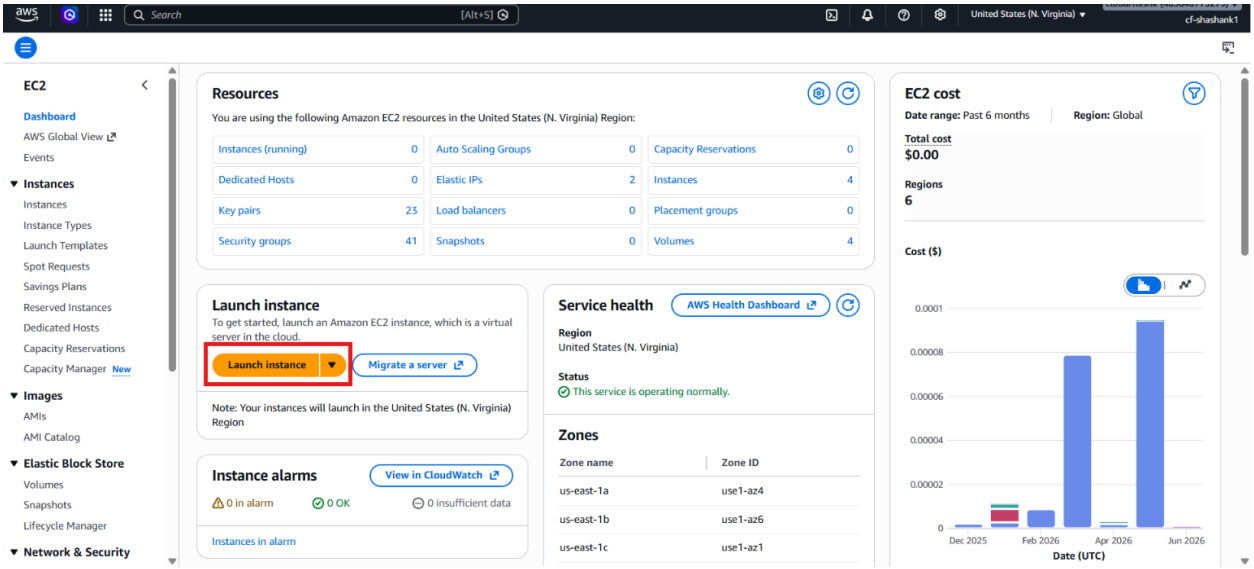


- Choose **Attach policies directly**.
- Select **S3PutObjectPermission**.
- Add the permission.

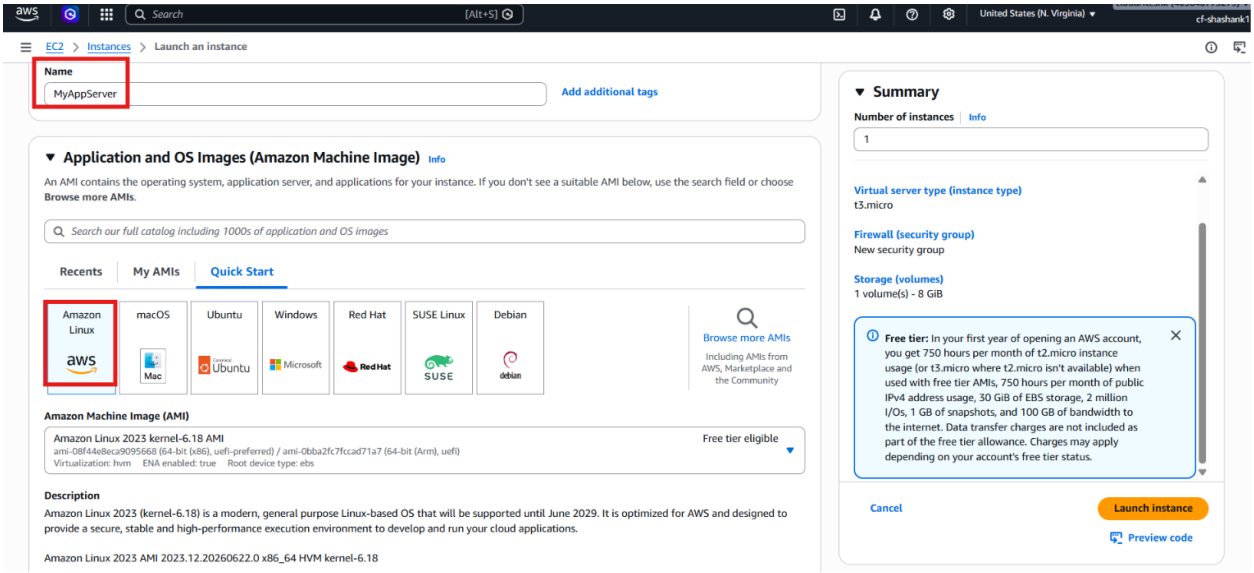


Step 5 – Launch the EC2 Instance for the Insecure Version

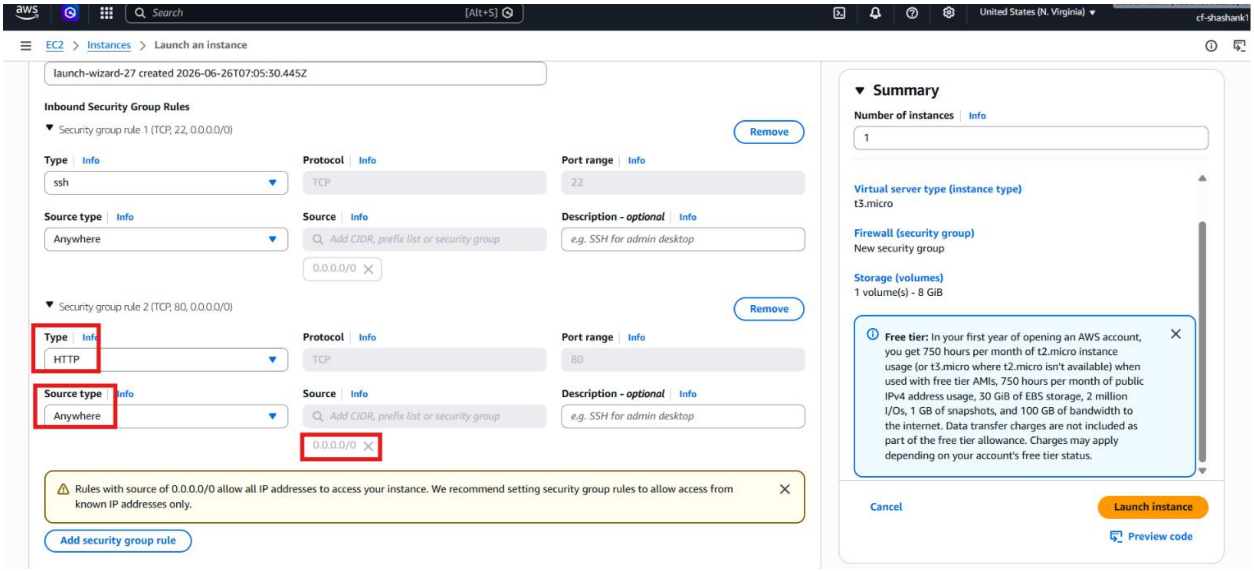
- Open **Amazon EC2**.
- Click **Launch instance**.



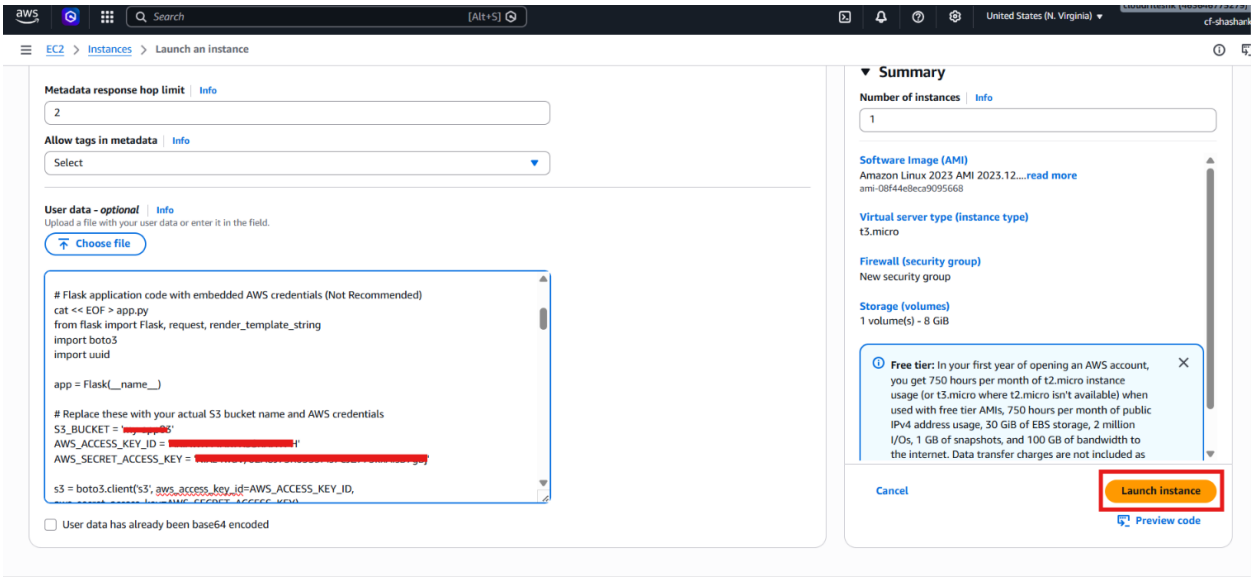
- Enter the instance name: MyAppServer and choose the subnet and VPC.
- Select **Amazon Linux** and choose **t3.micro** instance and a **Key pair**.



- Allow HTTP traffic on port **80**.



- Use the provided user data script with hardcoded credentials and do change the credentials by your own credentials.
- Scroll down and click on **Advanced Details → User data** and paste the script.

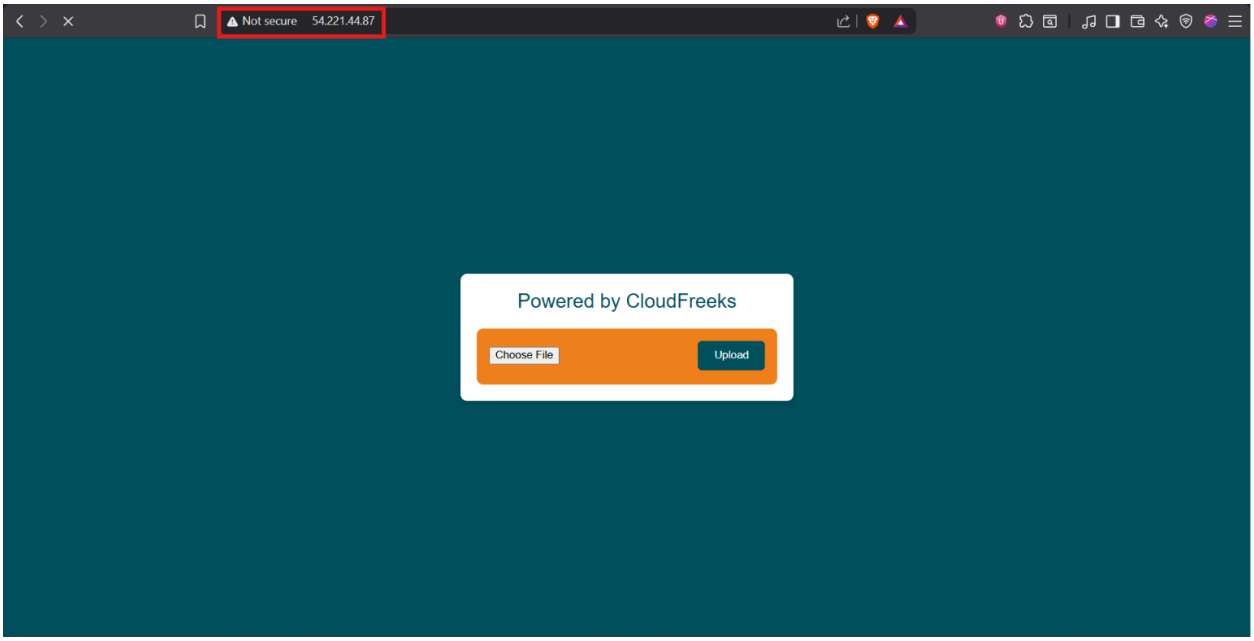


- Launch the instance.

Step 6 – Deploy the Flask App Using Access Keys

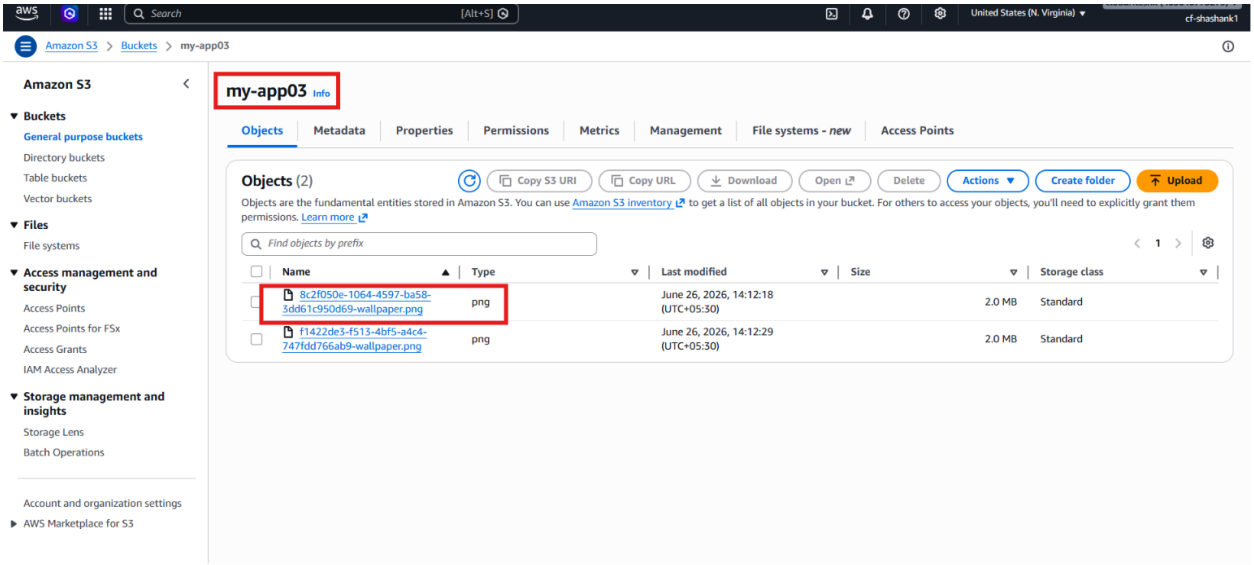
- Use the user data script that installs:
  - Python
  - pip
  - Flask
  - Gunicorn
  - Boto3
- Create the Flask app under /var/www.

- Configure the app to use:
    - the S3 bucket name
    - the IAM user access key
    - the IAM user secret key
- Start the systemd service.
- Open the EC2 public IP in a browser.



- Upload a file through the app.

- Verify that the file appears in the S3 bucket.

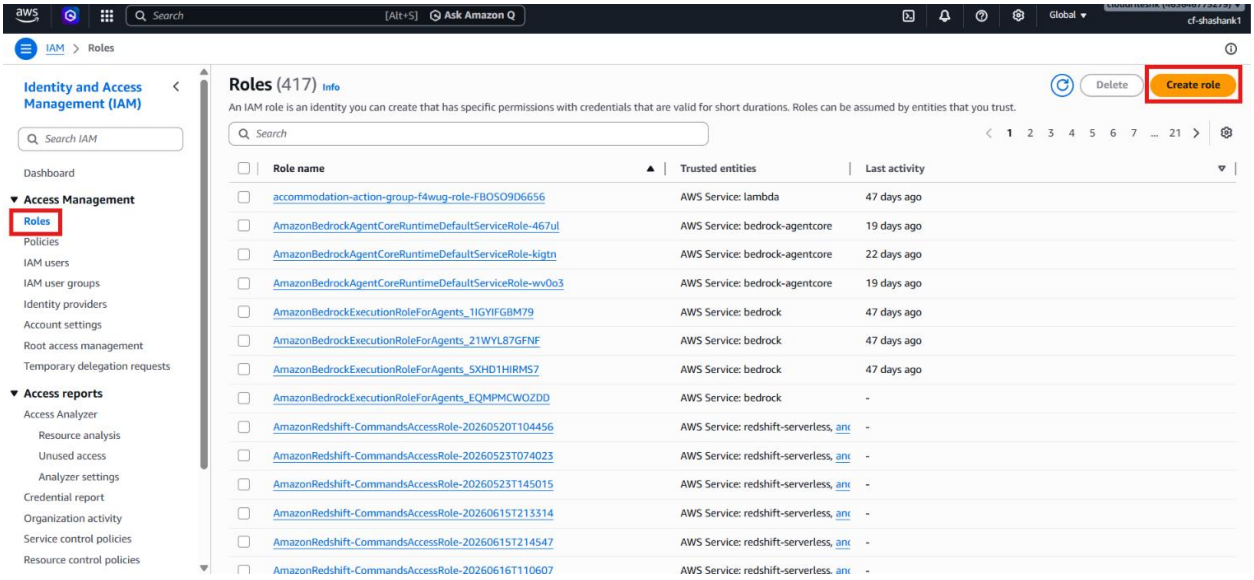


Step 7 – Observe Why the First Method Is Not Recommended

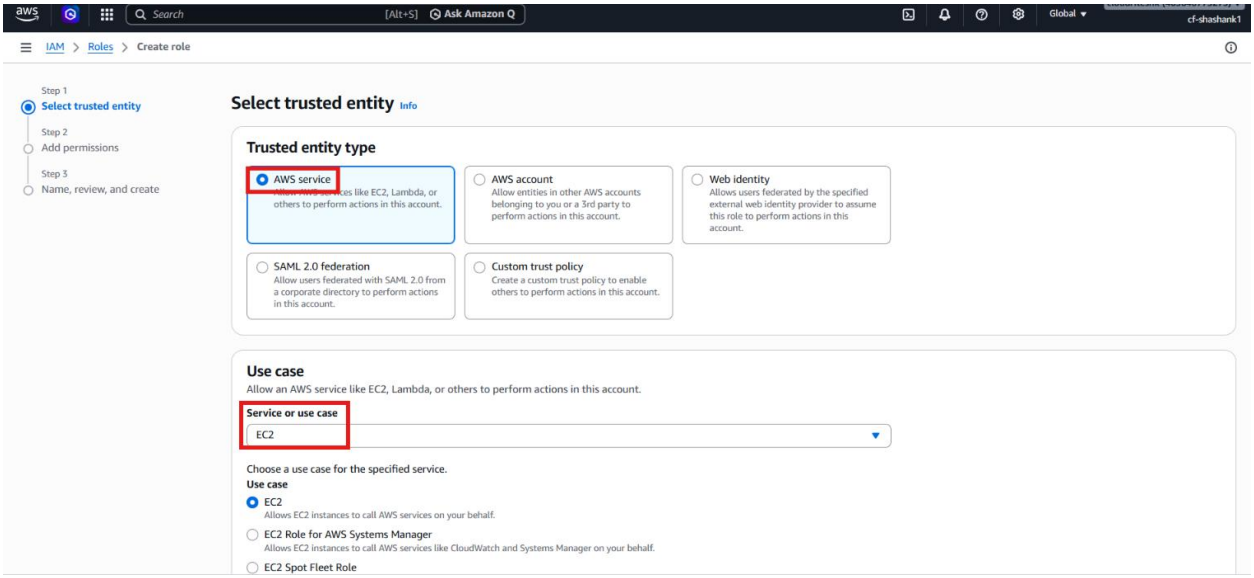
- Review the Flask code.
- Notice that the access key and secret key are embedded directly in the app.
- Understand that this creates a security risk because anyone with server access can steal the credentials.
- Treat this method only as a demonstration of the insecure pattern.

Step 8 – Create the IAM Role for the Secure Version

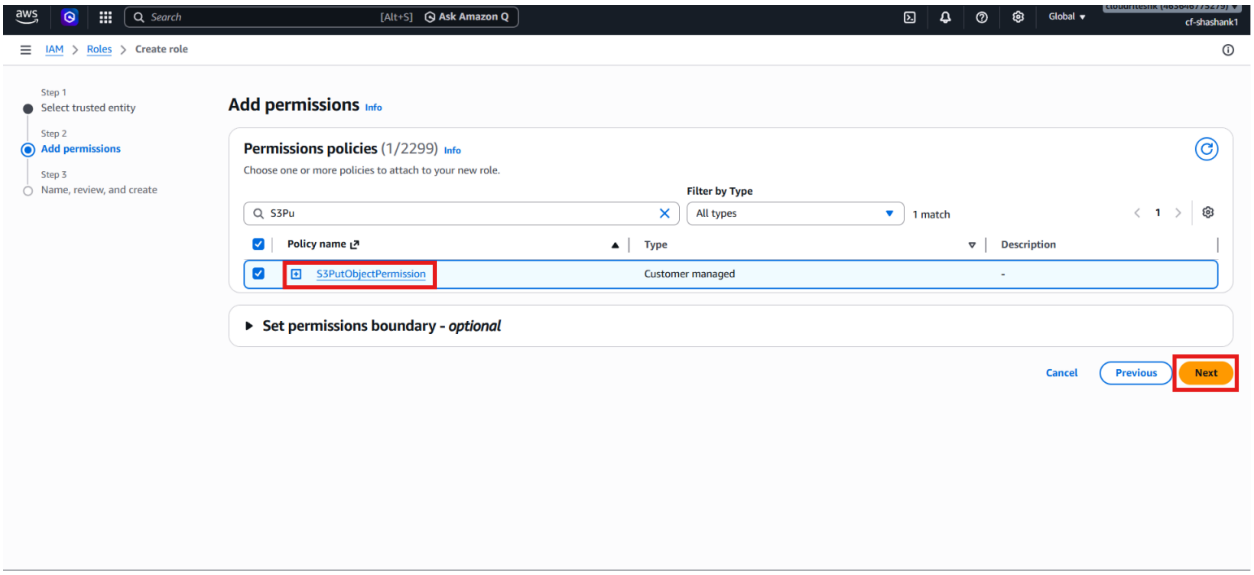
- Open **IAM → Roles**.
- Click **Create role**.



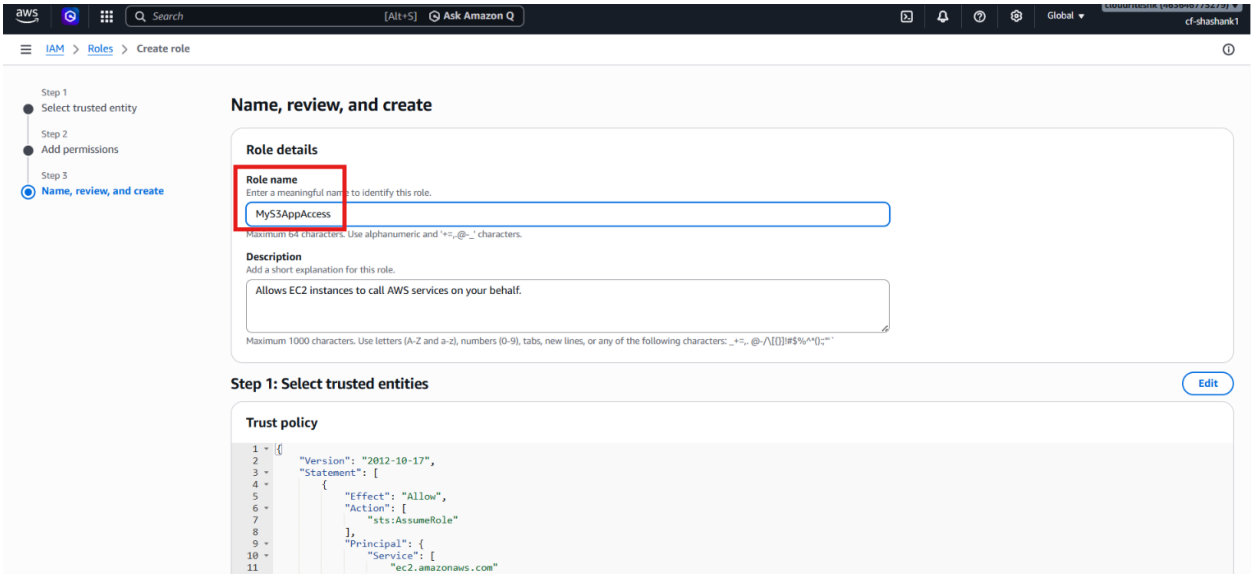
- Choose **AWS service**.
- Select **EC2** as the trusted service.



- Attach the same S3 upload policy.

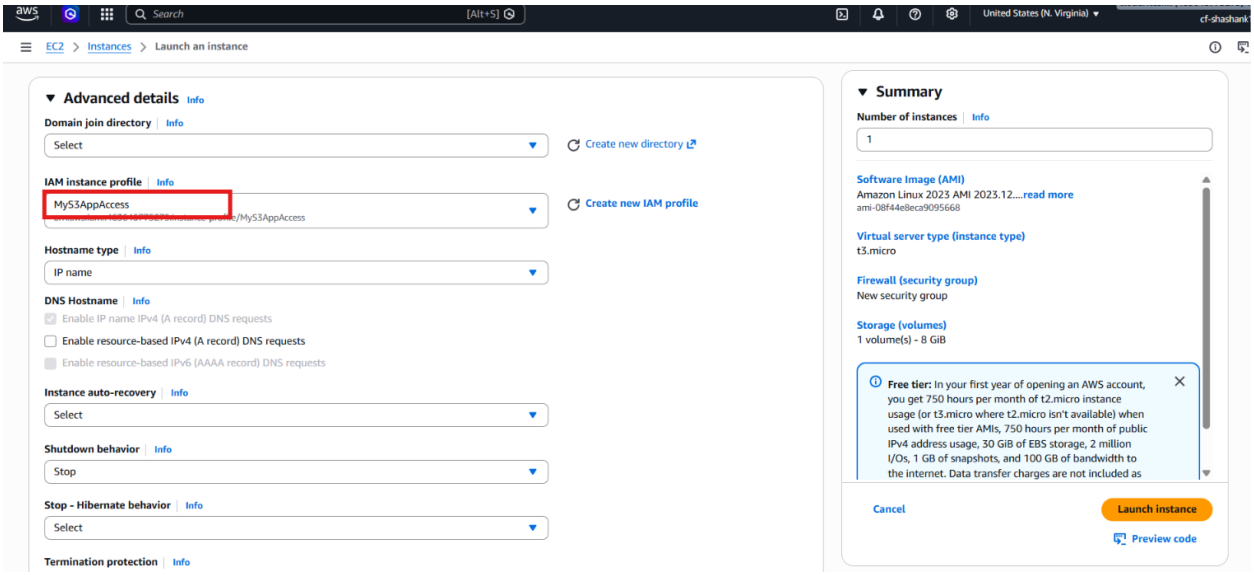


- Enter the role name: MyAppS3Access
- Create the role.

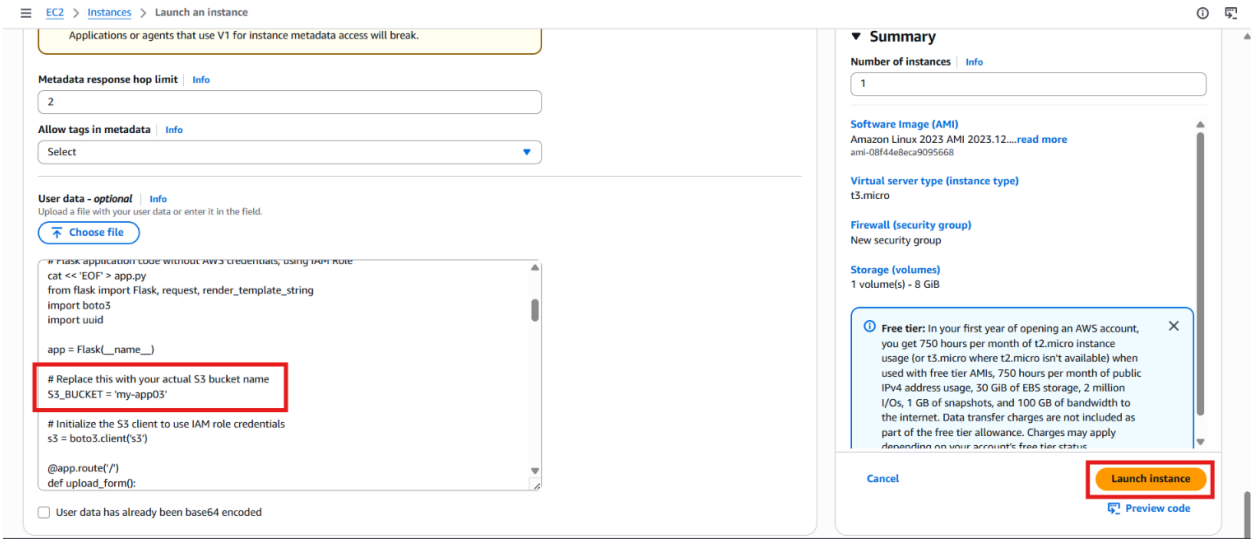


Step 9 – Launch the EC2 Instance with the IAM Role

- Launch a new EC2 instance.
- Keep the same OS and instance type.
- Add port **80** for the web application.
- Attach the **MyAppS3Access** IAM role to the instance.



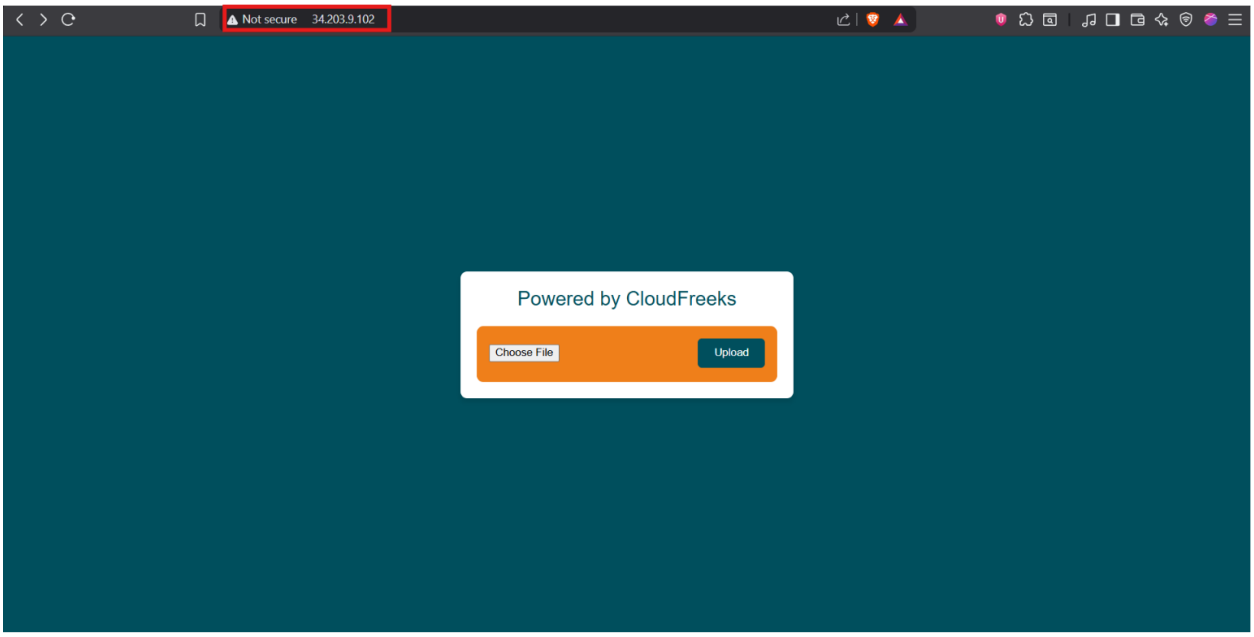
- Use the secure user data script.
- Launch the instance.



- Wait for the instance to become ready.

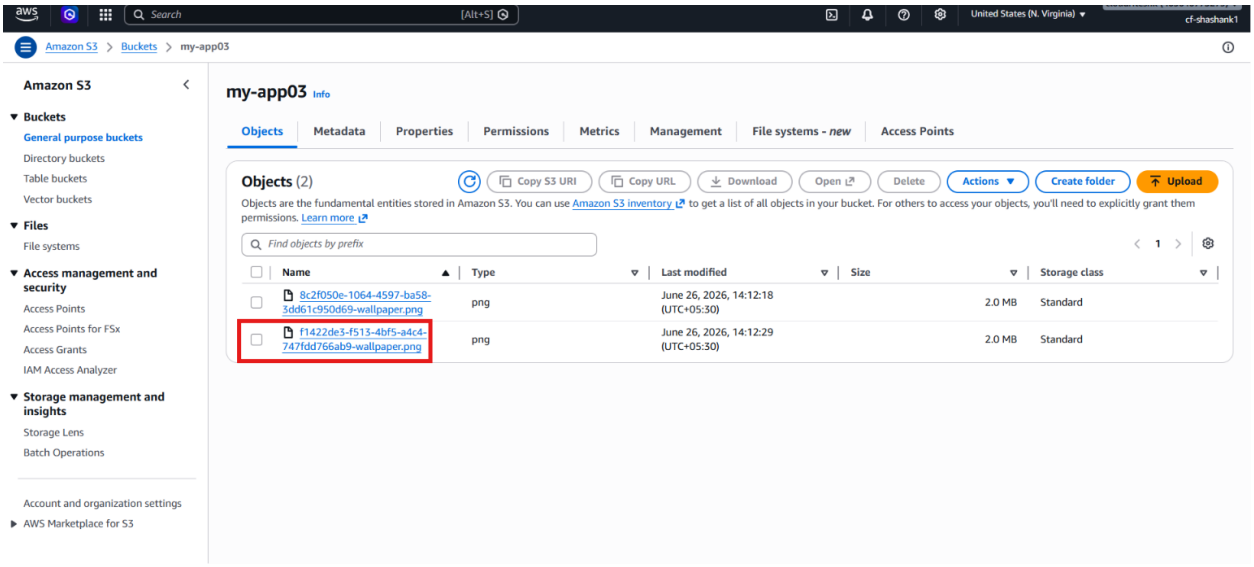
Step 10 – Deploy the Secure Flask App

- Use the user data script that does **not** contain access keys.
- Keep only the bucket name in the code.
- Open app by opening the **Public IPv4 Address** of instance.
- Open the application in a browser.



- Upload a file again.

- Verify that the file is stored in the S3 bucket successfully.



Step 11 – Compare the Two Approaches

- Compare the insecure script and the secure script.
- Confirm that the insecure script stores AWS keys in code.
- Confirm that the secure script uses the IAM role instead.
- Verify that both methods can upload files, but only the role-based method follows AWS best practices.

Verification

- The S3 bucket was created successfully.
- The IAM user and S3 upload policy were created successfully.

- The insecure Flask app uploaded a file using embedded credentials.
- The IAM role was created for EC2.
- The secure Flask app uploaded a file without hardcoded AWS credentials.
- The uploaded file appeared in the S3 bucket.
- The role-based setup matched the recommended AWS security practice